MODULE 1 — LAB: HUNT THE FRONT DOOR BY HAND

> **Level coverage: L0 → L1.** Budget: 40 min. Artifacts: A1, A2, A3 (all Tier 1).
> Deliverables: **H-D01**, **H-D02** in your ATHF workspace.
> All data is in `index=thrunt`. Your pod credentials are in the shared spreadsheet.

SETUP (5 MIN)

Initialize your workspace if you haven't:

```
athf init
```

`athf init` asks five quick questions (workspace name, SIEM, EDR, hunt-ID prefix, retention). **Just press Enter through all of them** — the defaults (Splunk, `H-`, etc.) are exactly what this workshop expects. This creates `hunts/`, `AGENTS.md`, and the hunt template. Confirm it worked:

```
athf hunt list        # you'll see 3 seeded example hunts (H-0001–H-0003) — that's expected
```

`athf init` ships three example hunts so the workspace isn't empty on day one. Leave them; you'll add your own two (H-D01, H-D02) alongside them in this module.

PART A — FIND THE FRONT DOOR (LO, BY HAND) (12 MIN)

You've been told developers install build tooling from a public registry. Start in the Splunk search bar. No hunt record yet — hunt like you always have.

Guiding question (orientation): *Developers install tooling all day. Before you search — what would a **malicious** install do that a normal one wouldn't? Name the behavior you're looking for, not the package.*

Step 1. Look for the install itself:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*packmule-cli@2.4.1*" OR properties.command_line="*postinstall.js*")
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Answer key — staff + self-study

Expected: `packmule-cli@2.4.1` being installed on `WS-BUILD-01` (user alice.chmod),
with a `node postinstall.js` child. That's **A1** — front door found.
Model answer (orientation Q): the behavior to name is "a package post-install script that does something beyond installing" — you don't yet know what, but a

Answer key — staff + self-study (cont.)

post-install spawning a child process is the thing to chase.

Step 2. What did the post-install script *do*? Pivot to its children:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01"
properties.parent_image_path="*\\node.exe" properties.image_path="*\\cmd.exe"
| table _time hostname principal properties.parent_image_path properties.command_line
```

Scoped to `WS-BUILD-01` — the build host you were told about. Staying on that one
host now is what makes Module 2's host-agnostic pivot a genuine discovery.

Answer key — staff + self-study

Expected: a `node.exe → cmd.exe` process-create chain. A post-install script spawning an interactive shell is the behavior that matters. That's **A2**.

Step 3. What did that shell run? The recon burst:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01"
properties.image_path IN ("*\\whoami.exe","*\\net.exe","*\\ipconfig.exe") properties.parent_image_path="*\\cmd.exe"
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Guiding question (interpretation): *You now have three events. What is the single behavioral story that connects them — and why does it matter that it's a signed binary with a clean hash?*

Answer key — staff + self-study

Expected: `whoami /all`, `net group "Domain Admins" /domain`, `ipconfig /all`
within ~2 min of install. That's **A3**.
Model answer (interpretation Q): the story is install → post-install script → shell → host/domain recon — one lineage. It matters because the binary is signed

Answer key — staff + self-study (cont.)

with a clean hash and no CVE, so indicators are useless; only the *behavior* (trusted dev process spawning a shell that recons) betrays it. This is the shape you'll reuse in every later module.

PART B — THE DEAD-END (FEEL THE LO PAIN) (5 MIN)

Guiding question (reflection): *Without scrolling up — answer these out loud with your pod. If you can't, that's the lesson.*

- What were the exact three queries you just ran?
- Which hosts did you rule *out*?
- If a teammate sat down right now, could they reproduce your path?

Answer key — staff + self-study

Model answer (reflection Q): most pods can't reproduce all three queries from memory — that IS the L0 wall. You found real malicious activity and none of it is
captured anywhere but short-term memory and browser history. In a real hunt you'd
run 20+ queries; by query 15 you can't remember what query 6 ruled out. This is

Answer key — staff + self-study (cont.)

why hunts get lost and re-run. The fix is Part C — write it down.

PART C — WRITE IT DOWN (L1) (15 MIN)

Guiding question (decision): *You found one confirmed chain and one loose thread. Which becomes a "confirmed" hunt and which becomes an "open" one — and why is writing down the open one just as important?*

Convert what you found into two durable hunts.

H-D01 — post-install execution chain:

```
athf hunt new \
  --title "Trojanized build-CLI post-install spawns shell + recon" \
  --technique T1195.002 \
  --tactic execution --tactic initial-access \
  --platform windows \
  --data-source edr \
  --actor "CRON GOBLIN" \
  --behavior "package-manager post-install script spawns an interactive shell that runs host/domain recon" \
  --location "developer workstations (WS-BUILD-*, WS-DEV-*)" \
  --evidence "Sysmon/eCAR process create: node.exe → cmd.exe → whoami/net/ipconfig" \
  --threat-context "packmule-cli@2.4.1 backdoored via compromised publish token; signed binary, clean hash, no CVE" \
  --hunter "<your-name>" \
  --non-interactive
```

Open the created file in `hunts/` and fill the **Check** and **Keep** sections: paste the three SPL queries, the hosts you confirmed, and the verdict.

> **On "verdict":** it isn't a literal field name — it's the record's **Status** (`planning` → `completed`) plus the one-line finding in the **Keep / Findings** section (`confirmed` / `not-found` / `needs-follow-up`). When the slides say "update the verdict," that's the pair to update.
>
> **Answer key — staff + self-study**

> **On "verdict": (cont.)** it isn't a literal field name — it's the record's **Status**
>
> Fill Check/Keep with the three Part-A queries, confirmed host `WS-BUILD-01`, and
> verdict **confirmed**. Target shape: `HUNTS_EXPECTED/H-D01.md` — staff can show it.
> Model answer (decision Q): H-D01 is confirmed (you saw the full chain); H-D02 is
> open/needs-follow-up (beacon observed, not characterized). Writing the open one

> **On "verdict": (cont.)** it isn't a literal field name — it's the record's **Status**
>
> down matters because it's the deliberate thread Module 2 pulls — an honest "not
> done yet" is a real deliverable, not a gap.

H-D02 — unusual outbound after install. Before you write it, hunt it (this one's still Tier 1):

```
index=thrunt sourcetype="apache:access:json" source="*proxy_access.log"
server="cdn-packmule-telemetry.net"
| timechart span=5m count by client
```

Don't fully solve it — that's Module 2/3 territory. Just capture the *observation* as a hunt so it's on record:

```
athf hunt new \
  --title "Periodic outbound to CDN-lookalike after build-CLI install" \
  --technique T1071.001 \
  --tactic command-and-control \
  --platform windows \
  --data-source proxy \
  --actor "CRON GOBLIN" \
  --behavior "regular-interval HTTPS to a young CDN-lookalike domain following post-install execution" \
  --location "WS-BUILD-01 egress via PROXY-01" \
  --evidence "proxy access log: ~5-min-interval requests to cdn-packmule-telemetry.net" \
  --hunter "<your-name>" \
  --non-interactive
```

Mark H-D02's verdict **needs-follow-up**.

Answer key — staff + self-study

Expected: periodic (~5-min-interval) outbound from `WS-BUILD-01` to `cdn-packmule-telemetry.net`, a young CDN-lookalike domain. Beacon observed, not
yet characterized — verdict **needs-follow-up**. That honest open state is exactly
what Module 3 picks up (shared-infrastructure pivot to the second domain).

PART D — VERIFY (3 MIN)

```
athf hunt list          # H-D01 and H-D02 should both appear (plus any you already had)
athf hunt validate      # every hunt should pass structural validation
```

Flag a staffer for a spot-check. You should leave Module 1 with **H-D01 and H-D02 on disk** — one confirmed, one open. From a fresh `athf init` that's five hunts total: the three seeded examples (H-0001–H-0003) plus your two. Just confirm H-D01 and H-D02 are present and valid.

THE AGENTIC MOMENT (RECAP)

Guiding question (reflection): *You found A1–A3 by hand at L0. What specifically was lost the moment you'd have closed your laptop — and what did* `athf hunt new` *actually change?*

Manual dead-end: you found A1–A3 by hand, but your queries lived in tabs and your reasoning lived in your head — unreproducible, unshareable, gone when you close the laptop.

Agentic recovery (L1): `athf hunt new` turned the hunt into a durable record. It's not AI yet — it's the *substrate* AI needs.

Answer key — staff + self-study

Model answer (reflection Q): what's lost at L0 is everything except the finding itself — the queries, the ruled-out hosts, the reasoning chain. `athf hunt new` converts an in-your-head investigation into a reusable record. It isn't AI, but you
can't reuse, research over, or coverage-check something you never wrote down.

Answer key — staff + self-study (cont.)

Module 2 proves it by pivoting off H-D01 to find a victim you had no indicator for.

MODULE 1 — BONUS / STRETCH HUNTS

> For pods who clear H-D01 and H-D02 early. All still Tier 1–2, still in `index=thrunt`. Staff: point fast finishers here instead of letting them idle. See `seeding/TIER_MANIFEST.md` for the full answer key.

BONUS 1 — ENUMERATE EVERY INFECTED HOST (TIER 2 PIVOT)

Guiding question (decision): *You confirmed WS-BUILD-01. Was it the only one? Decide how to look for more victims WITHOUT anchoring on the host or package you already have.*

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\node.exe" (properties.image_path="*\\cmd.exe" OR properties.image_path="*\\sh.exe")
| stats count values(hostname) as hosts by principal
```

Answer key — staff + self-study

Expected: more than one workstation touched by the same install→shell pattern.
Model answer (decision Q): pivot on the *behavior* (node→shell), not the host or package. This is a soft preview of A4 / Module 2 — don't fully solve the second-wave host yet; that's the point of Module 2.

BONUS 2 — BASELINE THE NOISE (BEHAVIORAL RIGOR)

Guiding question (interpretation): *How often does `node.exe` **legitimately** spawn a shell in this environment? Run the pivot without the malicious filter — what does normal look like, and how would you defend "this is weird"?*

Baseline the node→shell rate across the whole environment and window — the same pivot as Bonus 1, minus any malicious anchor:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\node.exe" (properties.image_path="*\\cmd.exe" OR properties.image_path="*\\sh.exe" OR properties.image_path="*\\powershell.exe")
earliest=-30d@d latest=now
| bucket _time span=1d
| stats count as spawns dc(hostname) as hosts by _time
| sort 0 _time
```

Answer key — staff + self-study

Expected: outside the CRON GOBLIN window the daily `node→shell` count sits at
~0 across the fleet — legitimate build tooling almost never spawns an interactive shell — so the install-day burst on `WS-BUILD-01` stands out as a clear spike against an otherwise-flat baseline.

Answer key — staff + self-study (cont.)

Model answer (interpretation Q): run the pivot with no malicious filter and read the daily rate. A hunter who knows normal (near-zero node→shell) can defend "this
is weird"; a hunter who doesn't is guessing. Pure Pyramid-of-Pain practice — behavior is only anomalous relative to a baseline.

BONUS 3 — WRITE THE RULED-OUT HUNT

Guiding question (decision): *Find one thing that looked suspicious but was benign. Why is writing up a dead-end a real deliverable and not wasted effort?*

> **Answer key — staff + self-study**
>
> Expected candidate: the `PackmuleCacheCleanup` scheduled task (A6) — it looks like
> persistence, it isn't. Write it up as a hunt with verdict **not-found / benign**.
> Model answer (decision Q): a documented dead-end stops the next hunter re-running
> your dead query. In LOCK terms, a ruled-out hypothesis is a Keep — it tells the

Answer key — staff + self-study (cont.)

next person where NOT to look, which is as valuable as a positive finding.

MODULE 2 — LAB: REUSE THE BEHAVIOR, FIND THE HIDDEN VICTIM

> **Level coverage: L2.** Budget: 30 min. Artifact: A4 (Tier 2), with A5 as stretch.
> Deliverable: **H-D03**.
> Requires your Module 1 hunts (H-D01, H-D02) on disk.

PART A — ORIENT OFF YOUR OWN CORPUS (L2) (5 MIN)

Guiding question (orientation): *Before you query anything new — what has your own repo already told you, and which parts of the intrusion have you provably NOT looked at yet?* The point of L2 is that your last hunt tells you where to go next.

Ask your hunt repo what you've already done:

```
athf similar --hunt H-D01
athf hunt coverage --detailed
```

Now the free-text form, to feel how the corpus answers a behavioral question:

```
athf similar "post-install shell spawns recon child process"
```

Answer key — staff + self-study

Expected: coverage shows initial-access and execution have hunts; persistence,
command-and-control, credential-access, and exfiltration are empty. Those empty
rows ARE the rest of the intrusion — students close them across Modules 3–4.
Model answer (orientation Q): the repo already documents the node→shell→recon

Answer key — staff + self-study (cont.)

behavior (H-D01); the empty coverage rows are the explicit map of what's left. `athf similar` should rank H-D01 top against the free-text behavioral query.

PART B — THE DEAD-END (WHY KEYWORD SEARCH FAILS) (5 MIN)

Guiding question (decision): *You want to find other victims. Your instinct is to search the indicators you already have (host, package, date). Predict what that returns — and decide whether it can possibly teach you anything new.*

Try it the naive way — reuse the *indicators* from H-D01:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
hostname="WS-BUILD-01" properties.command_line="*packmule-cli@2.4.1*"
```

Now widen the time picker to **All time**, keeping the hostname.

Answer key — staff + self-study

Expected: one host. Widening to All time still returns only WS-BUILD-01.
Model answer (decision Q): anchoring on what you already saw only finds more of
what you already saw. This is the wall — CRON GOBLIN put the second victim somewhere your indicators don't reach, so the correct next move is to abandon

Answer key — staff + self-study (cont.)

indicators and pivot on behavior (Part C), not to keep tuning the keyword search.
Dead-end to expect: students keep adding OR-clauses to the keyword search hoping
to widen it. Widening indicators is still indicators — redirect them to behavior.

PART C — PIVOT ON BEHAVIOR, NOT INDICATORS (L2) (10 MIN)

Guiding question (orientation): *Take the shape you documented in H-D01 — trusted-dev-process → unexpected shell → recon. Before you run it host-agnostic and time-agnostic across the whole environment, what do you EXPECT to fall out that the keyword search couldn't reach?*

No hostname, no date, no package name:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\node.exe" (properties.image_path="*\\cmd.exe" OR properties.image_path="*\\sh.exe")
| table _time hostname principal properties.image_path properties.command_line
```

Now take each `cmd.exe` you find and look at *its* children across all hosts:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
properties.parent_image_path="*\\cmd.exe" properties.image_path IN ("*\\whoami.exe","*\\systeminfo.exe","*\\net.exe","*\\ipconfig.exe")
| stats min(_time) as first values(properties.image_path) as cmds by hostname principal
| eval first=strftime(first,"%Y-%m-%d %H:%M:%S")
```

> The `strftime` renders `first` as a readable date instead of raw epoch — so the
> ~25-hour gap between the two hosts is something you can see at a glance.

Guiding question (interpretation): *A second host appears. What specifically makes it the same intrusion and not a coincidence — and what did you have zero indicators for a moment ago?*

Answer key — staff + self-study

Expected: a second host — `carol.grep` on `WS-DEV-04`, roughly 25 hours after the
first — with the identical node→cmd→recon lineage. That's **A4**.
Model answer (interpretation Q): it's the same intrusion because the *behavioral shape* is identical (trusted dev process → shell → whoami/systeminfo/net), not

Answer key — staff + self-study (cont.)

because any indicator matched — host, date, and package all differ. You found a
victim you had no indicator for, purely because you wrote the behavior down in M1.
Hint trigger: if a pod is stuck >5 min here, give the host-agnostic pivot from staff/HINT_CARDS Module 2 Tier 3 — the `properties.parent_image_path="*\\node.exe"` query above.

PART D — WRITE H-D03 (10 MIN)

Guiding question (decision): *What is the ONE methodology note that makes this hunt reusable by the next hunter — the sentence that captures why keyword search failed and behavior won?*

```
athf hunt new \
  --title "Second-wave post-install recon on different host (behavioral pivot)" \
  --technique T1059 \
  --tactic execution --tactic discovery \
  --platform windows \
  --data-source edr \
  --actor "CRON GOBLIN" \
  --behavior "same post-install-shell-to-recon lineage as H-D01, time-shifted onto a different developer workstation" \
  --location "<the second host + user you found in Part C> — NOT the WS-BUILD-01 host from H-D01" \
  --evidence "Sysmon node.exe → cmd.exe → whoami/systeminfo, host-agnostic pivot" \
  --threat-context "found via athf similar + behavioral pivot off H-D01, not keyword search" \
  --hunter "<your-name>" \
  --non-interactive
```

Then: in the record, explicitly write the methodology note.

Answer key — staff + self-study

Fill the `--location` with the second host from Part C: `WS-DEV-04 (carol.grep)`.
Model answer (decision Q): the note is *"keyword/host search failed; the host-agnostic behavioral pivot from H-D01 surfaced this."* That single line is the
L2 lesson made durable — it tells the next hunter to pivot on shape, not indicators.

Answer key — staff + self-study (cont.)

Target shape: see `HUNTS_EXPECTED/H-D03.md`. Staff can show it if a pod is stuck.

PART E — VERIFY (OPTIONAL STRETCH TO A5) (AS TIME ALLOWS)

```
athf hunt list            # now 3 hunts
athf hunt coverage         # execution now has 2 hunts; persistence still empty
```

Guiding question (decision): *Persistence is still an empty coverage row. Reuse the exact same "unusual child of a trusted dev process" idea — where would you point it next?*

Stretch (A5, Tier 2 → sets up Module 3):

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*core.hooksPath*") OR (properties.parent_image_path="*\\cmd.exe" properties.command_line="*loader.js*--beacon*")
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Don't fully solve it — Module 3 does. Just notice the coverage gap is real and pointing you somewhere.

Answer key — staff + self-study

Expected: a `git config --global core.hooksPath …` → `pre-commit.cmd` → `node loader.js --beacon` chain on WS-BUILD-01 (T1546 git-hook persistence). Same "unusual child of a trusted dev process" shape, now aimed at git instead of npm.

Model answer (decision Q): point the reused behavior at git — the persistence

Answer key — staff + self-study (cont.)

mechanism re-launches the beacon on every developer commit. Leave it noticed, not
fully solved; Module 3's research step is what characterizes it. This is A5.

THE AGENTIC MOMENT (RECAP)

Guiding question (reflection): *Why did every indicator-anchored search miss the second victim, and what exactly did writing H-D01 down in Module 1 buy you here?*

Manual dead-end: the second victim was time-shifted and host-shifted; every keyword/time search anchored on H-D01's indicators missed it.

Agentic recovery (L2): `athf similar` + `athf hunt coverage` let you pivot on the *behavior you documented*, not the indicators you observed — and the documented behavior found a victim you had zero indicators for. Coverage also named your open gaps (persistence, C2, exfil), which is exactly what the Module 4 agent will read to propose your next move.

Answer key — staff + self-study

Model answer (reflection Q): indicator searches missed the second victim because
the actor changed every indicator (host, date, package) while keeping the behavior
constant — indicators sit at the bottom of the Pyramid of Pain and rotate freely.
Writing H-D01 down turned an in-your-head observation into a reusable behavioral

Answer key — staff + self-study (cont.)

query; without that record there is nothing to pivot on, and carol.grep/WS-DEV-04
stays invisible. Memory is what converted one finding into a repeatable hunt.

MODULE 2 — BONUS / STRETCH HUNTS

> For pods who finish H-D03 early. Tier 2. See `seeding/TIER_MANIFEST.md` for the answer key.

BONUS 1 — CLOSE THE PERSISTENCE GAP EARLY (A5, TIER 2)

Guiding question (decision): `athf hunt coverage` *shows persistence empty. You have a behavior that keeps working — "unusual child of a trusted dev process." Where do you point it to close persistence, without waiting for Module 3?*

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*core.hooksPath*") OR (properties.parent_image_path="*\\cmd.exe" properties.command_line="*loader.js*--beacon*")
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Answer key — staff + self-study

Expected: a `pre-commit` git hook re-launching a beacon loader on developer commit
(T1546) — the same "unusual child of a trusted dev process" shape aimed at git.
Model answer (decision Q): reuse the H-D01/H-D03 behavior against git instead of
the package manager. Write it as a hunt if found — a legitimate Module 2 finding

Answer key — staff + self-study (cont.)

that gives a head start on Module 3.

BONUS 2 — RULE OUT THE SIMILARITY FALSE-POSITIVE (RH-BENIGN-POSTINSTALL, RED HERRING)

Guiding question (interpretation): *A behavior-similarity search on H-D01's structure surfaces a second candidate. Same shape — but is it the same behavior? What single detail decides malicious vs. benign here?*

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*postinstall*" OR properties.command_line="*node-gyp*" OR properties.command_line="*npm install*")
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Answer key — staff + self-study

Expected: `ivy.gzip` installed `sharp@0.33.2` on `WS-DEV-08` — a legitimate native-module package whose post-install spawns `node-gyp rebuild` to COMPILE a
native addon. Same shape as A1/A2, but the child compiles instead of shelling out
to recon, and no beacon follows.

Answer key — staff + self-study (cont.)

Model answer (interpretation Q): the deciding detail is what the child *does* — compile (benign) vs. recon + beacon (malicious). Write it up with verdict **benign / ruled-out**. Core M2 lesson: similarity surfaces candidates, verification rules them out — the shape matches, the behavior doesn't. Documenting

Answer key — staff + self-study (cont.)

the dead end stops the next hunter re-running your query.

BONUS 3 — QUANTIFY THE TIME-SHIFT

Guiding question (interpretation): *You found a second wave in Part C. Compute the delay between the two waves — does that interval tell you anything durable about the actor's operational cadence?*

Compute `first-seen(second host) - first-seen(WS-BUILD-01)`.

Answer key — staff + self-study

Expected: roughly 25 hours between the WS-BUILD-01 wave and the WS-DEV-04 wave.
Model answer (interpretation Q): cadence is a *behavioral* indicator that survives
infrastructure rotation — worth documenting for the Module 5 pivot, because the
actor's ~1-day rhythm can transfer even when hosts, domains, and hashes all change.

MODULE 3 — LAB: LET RESEARCH CRACK THE BEACON

> **Level coverage: L3.** Budget: 30 min. Artifacts: A8 (Tier 3), with A5 persistence as the primary hunt. Deliverable: **H-D04**.
> Requires H-D02 (the open beacon thread) on disk. Uses your LLM provider — flag a staffer if `athf agent` / `research` errors.

PART A — GENERATE RESEARCH FROM THE OPEN THREAD (L3) (10 MIN)

Guiding question (orientation): *You have two open threads: an uncharacterized beacon (H-D02) and an empty persistence coverage row. Before you run research — what specifically don't you know how to look for yet? Point research at the gap in your own tradecraft.*

```
athf research new \
  --topic "HTTPS C2 beacon detection via TLS JA3 pivoting and shared infrastructure; git-hook persistence in developer environments" \
  --technique T1071.001 \
  --depth basic
```

> **If your provider is slow or offline:** add `--no-web-search`. If it errors entirely, flag a staffer — do not burn the module debugging keys.

Read the resulting `R-XXXX` doc in `.athf/research/`.

Guiding question (interpretation): *The doc will contain both real tradecraft AND plausible-but-invented specifics. How do you tell them apart — and what's your rule for when a research claim becomes a finding?*

Answer key — staff + self-study

Answer key — staff + self-study (cont.)

Expected: the doc surfaces two usable ideas — (1) **Infrastructure reuse / JA3 pivoting**: actors stand up new look-alike C2 domains on the *same hosting* they already control, so domains that share nothing in their name still share an IP block. (The research agent may frame this as **JA3/TLS-fingerprint pivoting** — the classic form of "same client, rotated domains." Hold that thought for the checkpoint below.) (2) **git-hook persistence**: event-triggered execution (T1546) that blends with developer workflow.

Answer key — staff + self-study (cont.)

Answer key — staff + self-study (cont.)

Model answer (interpretation Q): circle anything concrete (a specific domain, a JA3 string, a CVE) — the agent will confidently invent specifics. The rule: a research claim is a *hypothesis*, never a finding, until you see it in `index=thrunt`. In particular: research will likely tell you to pivot on a **JA3 fingerprint**. Check your data — this environment's Zeek `ssl.log` has no `ja3` field (`| fieldsummary` on `sourcetype="bro:ssl:json"` proves it). That's the lesson: the agent gives you *tradecraft*, but you adapt it to the telemetry you actually have. Here the same "reused infrastructure" idea lands on the **shared hosting IP block** instead of a TLS hash (Part C). The doc tells you what to look for; the data decides what's real.

PART B — HUNT PERSISTENCE WITH WHAT RESEARCH TOLD YOU (A5) (8 MIN)

Guiding question (decision): *Research pointed at git-hook persistence. You've hunted "unusual child of a trusted dev process" before. How does that same behavior apply to git?*

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*core.hooksPath*") OR (properties.parent_image_path="*\\cmd.exe" properties.command_line="*loader.js*--beacon*")
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Answer key — staff + self-study

Expected: `git config --global core.hooksPath …` → `pre-commit.cmd` → `node loader.js --beacon` on WS-BUILD-01. That's **A5** — persistence that re-launches
the beacon on every developer commit.
Model answer (decision Q): it's the same "unusual child of a trusted dev process"

Answer key — staff + self-study (cont.)

shape from Module 1, now aimed at git instead of the package manager. The behavior
transfers; only the trusted process changed (npm → git).

PART C — THE DEAD-END, THEN THE SHARED-INFRASTRUCTURE PIVOT (A8, TIER 3) (7 MIN)

Guiding question (decision): *Go back to H-D02's beacon. Your instinct is to search the same domain for more C2. Predict what that returns — then decide what research told you to do instead.*

The naive way — search the same domain, and resolve it to the IP that's actually hosting it:

```
index=thrunt sourcetype="bro:ssl:json" server_name="cdn-packmule-telemetry.net"
| stats count values(server_name) as domain by id.resp_h
```

One domain, hosted at `203.0.113.44`. But research told you actors *reuse infrastructure* — they stand up new look-alike domains on the same hosting block. Pivot on the neighboring addresses instead of the domain name:

```
index=thrunt sourcetype="bro:ssl:json"
| where cidrmatch("203.0.113.44/30", 'id.resp_h')
| stats count values(id.resp_h) as ips by server_name
```

Guiding question (interpretation): *A second domain falls out sharing the same hosting block. Why is shared infrastructure stronger evidence of "same actor" than shared domain reputation or request volume would be?*

Answer key — staff + self-study

Expected: searching the same domain returns one domain — a dead end. The infrastructure pivot surfaces a **second** domain, `assets-packmule-cdn.com` (`203.0.113.45`, the adjacent address), beaconing low-and-slow (six callbacks over days). That's **A8**, the tasking channel you'd never have found by volume or domain reputation.

Answer key — staff + self-study (cont.)

Model answer (interpretation Q): a shared hosting block links domains that share nothing else — it reveals the actor's *infrastructure footprint*, which they reuse across rotated C2 domains. Reputation and volume are trivial to change; the hosting infrastructure is sticky tradecraft. Research told you what to look for (originally framed as JA3 pivoting); you adapted it to the telemetry you actually have (no `ja3` field, so you pivoted on the IP block instead); the data confirmed it.

Answer key — staff + self-study (cont.)

The legit CDN decoy (`static.docs-cdn.io`) is hosted at `203.0.113.60` — outside the `203.0.113.44/30` block — so the infra pivot cleanly separates it from the C2 pair.

PART D — WRITE H-D04 (5 MIN)

Guiding question (decision): *This hunt came from research, not from your own data. What field makes that provenance durable, and why does linking it matter three modules from now?*

```
athf hunt new \
  --title "Git-hook persistence + infra-linked secondary C2 channel" \
  --technique T1546 \
  --tactic persistence --tactic command-and-control \
  --platform windows \
  --data-source edr --data-source network \
  --actor "CRON GOBLIN" \
  --behavior "pre-commit git hook relaunches beacon; secondary low-slow C2 domain shares the primary's hosting block (203.0.113.44/30)" \
  --location "WS-BUILD-01; egress to cdn-packmule-telemetry.net + <the second domain you surfaced in Part C>" \
  --evidence "Sysmon core.hooksPath→loader.js; Zeek ssl two look-alike domains on adjacent C2 IPs in the /30 you pivoted on" \
  --research "<R-XXXX from Part A>" \
  --hunter "<your-name>" \
  --non-interactive
```

Answer key — staff + self-study

Expected: `--location` contains both domains: `cdn-packmule-telemetry.net + assets-packmule-cdn.com`.
Model answer (decision Q): the `--research R-XXXX` link makes the hunt traceable
back to the research that motivated it — the reasoning chain becomes durable instead of a fact you forget by Module 5. Target shape: `HUNTS_EXPECTED/H-D04.md`.

PART E — VERIFY (AS TIME ALLOWS)

```
athf hunt list            # 4 hunts
athf research list        # ≥1 research doc
athf hunt coverage        # persistence + C2 now covered; cred-access + exfil still open
```

Answer key — staff + self-study

Expected: 4 hunts, ≥1 research doc, coverage now showing persistence + C2 covered.
The two still-empty rows — credential-access and exfil — are Module 4's target.
You won't find them by guessing; an agent will have to *name the gap* for you.

THE AGENTIC MOMENT (RECAP)

Guiding question (reflection): *The low-and-slow tasking channel (A8) is six events over days on a reputable-looking domain. Why was it invisible to manual hunting, and what exactly did the research agent contribute that you couldn't?*

Manual dead-end: the low-and-slow tasking channel (A8) is six events over days on a reputable-looking domain — invisible by volume, reputation, or keyword. You didn't know infrastructure pivoting was even the move.

Agentic recovery (L3): `athf research new` surfaced the infrastructure-reuse tradecraft (pitched as JA3/TLS-fingerprint pivoting); you adapted it to the telemetry you actually have — no `ja3` field, so you pivoted on the shared hosting IP block instead — and confirmed it in your own data. The agent taught you *what to look for*; you translated it to your environment; Splunk proved it real. And you linked the hunt back to the research (`--research`), so the reasoning chain is durable — not a fact you'll forget by Module 5.

Answer key — staff + self-study

Model answer (reflection Q): A8 was invisible manually because there's nothing to
key on — low volume, good reputation, no matching indicator, and you didn't know
infrastructure pivoting existed. The agent contributed the *technique* (what to look for), not
the finding — you still confirmed it in `index=thrunt` and linked the hunt back to

Answer key — staff + self-study (cont.)

the research (`--research`), so the reasoning chain is durable. The agent taught;
the data proved.

MODULE 3 — BONUS / STRETCH HUNTS

> For pods who finish H-D04 early. Tier 2–3. See `seeding/TIER_MANIFEST.md`.

BONUS 1 — CHARACTERIZE THE BEACON STATISTICALLY

Guiding question (interpretation): *You confirmed periodicity by eye. What statistic would turn "looks periodic" into "is a beacon" — and what value range proves it?*

```
index=thrunt sourcetype="apache:access:json" source="*proxy_access.log"
server="cdn-packmule-telemetry.net"
| sort 0 _time | streamstats current=f last(_time) as prev by client
| eval delta=_time-prev | stats count avg(delta) p50(delta) stdev(delta) by client
```

Answer key — staff + self-study

Expected: a tight mean with small stdev around ~300s — the beacon signature.
Model answer (interpretation Q): inter-arrival delta with low stdev is the proof. A tight, regular interval (~5 min) with minimal jitter is what separates "looks periodic" from "is a beacon" — human traffic is bursty and irregular.

BONUS 2 — ASK RESEARCH A BAD QUESTION ON PURPOSE

Guiding question (reflection): *Run research on a CVE that doesn't exist. What does the agent do with a false premise — and how did you catch it?*

```
athf research new --topic "does CRON GOBLIN use CVE-2024-99999 for initial access"
```

Answer key — staff + self-study

Expected: the agent will often reason confidently about the fabricated CVE as if
it were real. That's the point.
Model answer (reflection Q): an agent will confidently elaborate on something invented — the most valuable 3 minutes in the module for building healthy

Answer key — staff + self-study (cont.)

skepticism. You catch it by checking the concrete claim against ground truth (`index=thrunt`, real CVE databases). Document how you caught it. Same rule as
Part A: a research claim is a hypothesis until the data confirms it.

BONUS 3 — RULE OUT THE LEGIT CDN (RED HERRING RH-LEGIT-CDN)

Guiding question (decision): *Not everything hitting a CDN-looking domain is C2. Find the benign CDN traffic and state — in one line — the exact discriminator that separates it from A7/A8.*

Answer key — staff + self-study

Expected: benign CDN traffic (`static.docs-cdn.io`, hosted at `203.0.113.60`) exists in the data alongside the malicious beacon.
Model answer (decision Q): the discriminator is hosting IP block + periodicity + domain age —
the benign CDN sits *outside* the `203.0.113.44/30` C2 block, won't beacon at a tight interval, and

Answer key — staff + self-study (cont.)

isn't a freshly-registered lookalike. (Note: JA3/TLS-fingerprint would be the classic discriminator,
but this environment's Zeek `ssl.log` has no `ja3` field — so the hosting-block pivot is the one that
actually runs here.) If you can state the discriminator in one line, you understand the hunt. Write it
as a benign/ruled-out hunt note.

MODULE 4 — LAB: LET THE AGENT FIND YOUR BLIND SPOTS

> **Level coverage: L4.** Budget: 30 min. Artifacts: A9, A10 (both Tier 3).
> Deliverable: **H-D05**.
> Requires your four hunts (H-D01..H-D04) on disk — the agent reasons over them. Requires an MCP-capable client + working provider.

PART A — CONNECT YOUR MCP CLIENT, THEN TALK TO YOUR REPO (L4) (5 MIN)

Point your MCP client (Claude Code is the supported default) at `athf-mcp-server`. Staff have the copy-paste config snippet for your provider; other MCP clients use their own config UI. Confirm the handshake — the agent now has these ATHF tools available: `generate_hunt_hypothesis`, `search_hunts_by_text`, `find_similar_hunts`, `list_coverage_gaps`, `propose_next_query`.

> **If the handshake fails:** flag a staffer. MCP config is fiddly and it's not worth burning the module on. Staff can also run the tools CLI-side as a fallback.

Guiding question (orientation): *You're about to ask your repo a question in plain English instead of calling a tool by name. What do you expect the agent to do with "What have I hunted before?" — and what does it mean that you don't pick the tool?*

```
What have I hunted before?
```

Answer key — staff + self-study

Expected: the agent reads your corpus and summarizes your four hunts (H-D01–H-D04)
— initial-access, execution, persistence, C2 — back to you.
Model answer (orientation Q): the agent picks the tool (`search_hunts_by_text` / `find_similar_hunts`) from your plain-English question. The point of L4 is that you

Answer key — staff + self-study (cont.)

drive it like a hunting partner, not a command line — the agent is now reading *your* repo.

Fallback (only if the NL question misfires or you're offline): the equivalent explicit call is `Use search_hunts_by_text to find my hunts about "beacon"`. Staff can also run `athf similar` / `athf hunt coverage` CLI-side — same data, no agent.

PART B — THE DEAD-END (YOU DON'T KNOW WHAT YOU'RE MISSING) (5 MIN)

Guiding question (decision): *You've found initial access, execution, persistence, C2. Are you done? Decide with your pod — and notice WHY this is the hardest kind of gap to see.*

Prove the trap to yourself — try to find exfil by guessing:

```
index=thrunt sourcetype="bro:dns:json" | stats count by query | sort -count
```

Answer key — staff + self-study

Expected: thousands of normal DNS queries, nothing obvious — a wall.
Model answer (decision Q): at L0–L2 you'd say "done" — there's no open thread,
nothing screaming "look here." That's the trap. The two remaining artifacts (credential theft, exfil) leave NO loose end pointing at them. A hunter without

Answer key — staff + self-study (cont.)

memory stops here and misses half the intrusion. Without knowing what exfil looks
like or *that you're even missing it*, guessing at DNS is hopeless.

PART C — LET THE AGENT NAME THE GAP (L4) (10 MIN)

Guiding question (decision): *Ask your repo the question every hunter should ask and almost never can. What is that question — and why can the agent answer it when you couldn't?*

```
Based on what I've already hunted, what should I hunt next?
```

Then push further, still in plain English:

> Give me a starting query for the credential-access and exfiltration gaps.

Answer key — staff + self-study

Expected: the agent reads your four hunts, maps them to the kill chain, and reports
the uncovered phases — **credential-access** and **exfiltration**. The invisible is now named. `propose_next_query` hands you starting queries for both.
Model answer (decision Q): the question is "based on what I've hunted, what should

Answer key — staff + self-study (cont.)

I hunt next?" The agent can answer because Modules 1–3 *wrote the hunts down* — it
reasons over your corpus and maps coverage to the kill chain. Your own memory
answered a question your intuition couldn't.

Answer key — staff + self-study (cont.)

Fallback (NL misfires / offline): the explicit calls are `Use list_coverage_gaps` and `Use propose_next_query focused on credential-access and exfiltration`. CLI equivalent from staff: `athf hunt coverage --detailed`.

TAKE THE PROPOSED DIRECTION AND HUNT THE TWO TIER-3 ARTIFACTS.

Credential access (A9):

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*type*\\.env*" OR properties.command_line="*\\.aws\\credentials*" OR properties.command_line="*findstr*password*.netrc*")
| table _time hostname principal properties.parent_image_path properties.image_path properties.command_line
```

Answer key — staff + self-study

Expected (A9): the beacon reading `.env`, `~/.aws/credentials`, `.netrc` on WS-BUILD-01 — credential theft from developer secret files.

DNS-tunnel exfil (A10) — the hardest artifact of the campaign:

```
index=thrunt sourcetype="bro:dns:json" qtype_name=TXT query="*.exfil-packmule.net"
| eval label=mvindex(split(query,"."),0) | eval len=len(label)
| stats count avg(len) as avg_label_len by id.orig_h
```

Guiding question (interpretation): *These two hits had no open thread pointing at them. Now that you see them, what makes A10 (DNS-tunnel exfil) the hardest artifact of the whole campaign?*

Answer key — staff + self-study

Expected (A10): high-entropy, long TXT subdomains to `exfil-packmule.net` from
WS-BUILD-01 — encoded exfil hiding in resolver noise.
Model answer (interpretation Q): A10 is hardest because encoded exfil hides inside
normal DNS — high average label length + entropy is the only tell, and it's buried

Answer key — staff + self-study (cont.)

in thousands of legitimate queries. Nothing pointed at it; only the agent naming
the exfil *phase* made it worth looking for at all.

PART D — WRITE H-D05 (5 MIN)

```
athf hunt new \
  --title "Credential access + DNS-tunnel exfil (coverage-gap driven)" \
  --technique T1552.001 \
  --tactic credential-access --tactic exfiltration \
  --platform windows \
  --data-source edr --data-source network \
  --actor "CRON GOBLIN" \
  --behavior "beacon reads .env/.aws/.netrc, then exfiltrates via encoded TXT DNS queries to a lookalike domain" \
  --location "WS-BUILD-01; DNS egress to exfil-packmule.net" \
  --evidence "Sysmon cred-file reads; Zeek dns high-entropy TXT subdomains" \
  --threat-context "found via MCP list_coverage_gaps naming uncovered phases + propose_next_query" \
  --hunter "<your-name>" \
  --non-interactive
```

Answer key — staff + self-study

Model answer: the `--threat-context` records the L4 lesson — this hunt exists
because an agent read your repo and named a gap you couldn't see. Target shape:
`HUNTS_EXPECTED/H-D05.md`.

PART E — VERIFY (3 MIN)

```
athf hunt list        # 5 hunts
athf hunt coverage     # ALL SIX phases now covered
if [ -f .athf/session.log ]; then cat .athf/session.log; else echo "(no session log yet — only created if you ran the CLI 'athf agent --workshop'; the MCP path in this module doesn't write one)"; fi
```

You've now reconstructed the entire first campaign — six kill-chain phases, five documented hunts. **Keep this workspace open.** In Module 5, you pivot to a second campaign from the same actor — and this repo becomes your head start.

PART F — ASK YOUR REPO ANYTHING (2 MIN, OPEN-ENDED)

Guiding question (reflection): *You have a five-hunt corpus and an agent that can read it. Ask it a real question in your own words — then notice what just changed about what your hunts ARE.*

No script for this one. Some starters, but go off-script:

- *"Which of my hunts share infrastructure?"*
- *"If a teammate inherited this repo, what would they need to re-run?"*
- *"What's the weakest hunt I've written and why?"*
- *"Summarize this whole intrusion in three sentences for my IR lead."*

Answer key — staff + self-study

Model answer (reflection Q): the L4 payoff in one move — your hunts stopped being
files and became something you can *interrogate*. Any of these questions works;
what changed is that the corpus is now a queryable memory, not a folder. That's the
muscle Module 5 makes you use under pressure.

THE AGENTIC MOMENT (RECAP)

Guiding question (reflection): *Credential access and exfil left no open thread — nothing pointed at them. What made them findable, and why could the agent only do this now and not in Module 1?*

Manual dead-end: credential access (A9) is buried in normal file activity and DNS exfil (A10) is buried in resolver noise — but the real wall is that *nothing pointed at them.* No open thread. At L0–L2 you stop, believing you're done.

Agentic recovery (L4): `list_coverage_gaps` read your five hunts, mapped them to the kill chain, and named the two phases you'd never covered — turning an invisible gap into a to-do list.

Answer key — staff + self-study

Model answer (reflection Q): they became findable because the agent named the
phase (credential-access, exfil) even though no evidence thread pointed there —
turning an invisible gap into a to-do list, with `propose_next_query` handing you
the start. The agent could only do this because Modules 1–3 wrote the hunts down;

Answer key — staff + self-study (cont.)

in Module 1 there was no corpus to reason over. Memory is what made the blind spot
visible.

MODULE 4 — BONUS / STRETCH HUNTS

> For pods who finish H-D05 early. Tier 3. See `seeding/TIER_MANIFEST.md`.
> This is also the ideal warm-up for Module 5 — the muscles are identical.

BONUS 1 — DECODE THE EXFIL VOLUME

Guiding question (interpretation): *You confirmed the DNS-tunnel channel exists. Estimate how much data left — what method gets you a bytes-per-query figure, and why is the method more important than the exact number?*

Answer key — staff + self-study

Expected: count the TXT queries × average encoded-label length, reason about bytes-per-query. You won't get an exact figure (it's synthetic).
Model answer (interpretation Q): quantify from label entropy + query count. The method matters more than the number because it's exactly what you'd do on a real

Answer key — staff + self-study (cont.)

incident to scope the loss — the synthetic total is arbitrary, the technique isn't.

BONUS 2 — ASK THE AGENT TO CONNECT THE WHOLE CAMPAIGN

Guiding question (reflection): *With all five hunts on disk, ask the agent for the single campaign narrative. What story should it stitch, and why is this a preview of the Module 5 payoff?*

```
Use find_similar_hunts across my whole corpus and tell me the single campaign narrative that links them.
```

Answer key — staff + self-study

Expected: the agent stitches H-D01→H-D05 into one CRON GOBLIN story — install →
recon → persistence → C2 → cred theft → exfil.
Model answer (reflection Q): a good agent narrates the full kill chain from your corpus. This previews Module 5: the repo *remembers the actor*, so the next campaign is a pivot, not a cold start.

BONUS 3 — PRE-STAGE FOR MODULE 5

Guiding question (decision): *Run coverage and screenshot it. Which two hunt structures should you know cold before you pivot to the Module 5 campaign — and why those two?*

```
athf hunt coverage --detailed
```

Answer key — staff + self-study

Model answer (decision Q): know your C2 hunt (H-D04) and credential-access hunt
(H-D05) structures cold — M5-A3 (shared-infra C2) and M5-A2 (runner cred theft)
reuse them directly. The fastest hunters in Module 5 are the ones already ready;
being ready is the whole point of having memory.

MODULE 5 — LAB: SAME ACTOR, NEW TTP

> **Level coverage: L0–L4 (your choice).** Budget: 25 min. Artifacts: M5-A1 (T1), M5-A2 (T2), M5-A3 (T3 ★), M5-A4 (T2). Deliverable: **H-D06**.
> No hand-holding by design. A second campaign — CI-runner telemetry from the same actor — is sitting in `index=thrunt`. Your Campaign-1 hunt repo is your head start.

THE RULES

- **No prescribed sequence.** Pick your approach across all five levels. The suggestions below are a fast route, not a script.
- **Reuse aggressively — just ask your repo.** Talk to your corpus in plain English: *"Have I seen this actor before?"*, *"Does this runner beacon match anything I've already hunted?"*, *"Which of my Campaign-1 hunts apply to a CI runner?"* (`athf similar` / `find_similar_hunts` are the explicit fallback.)
- **One hunt minimum (H-D06).** More if you're fast. Cite the earlier hunt you reused.
- **Stuck? Flag a staffer.** Module 5 has tiered hints too. Using them beats staring.

Guiding question (orientation): *Same actor, new battlefield. Before you pivot to the new campaign — what did CRON GOBLIN do in Campaign 1 that they'd likely do again, and which of your six documented hunts is your best head start?*

Answer key — staff + self-study

Model answer (orientation Q): CRON GOBLIN reuses behavior — post-install/injected
execution, credential theft, and (critically) C2 infrastructure. Your best head start is H-D04 (the C2 / shared-infra hunt): if the runner beacons to the same domain, your
Campaign-1 C2 hunt solves half of Campaign 2 instantly. H-D05 (cred access) is the

> **Answer key — staff + self-study (cont.)**
>
> second-best reuse. The whole module is a test of whether your repo remembers.

FAST ROUTE (IF YOU WANT A STARTING THREAD)

1. The obvious front door — malicious CI build step (M5-A1, Tier 1):

Guiding question (interpretation): *A CI runner executes build steps all day. What makes one step malicious rather than a normal pipeline job?*

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*curl*|*sh*" OR properties.command_line="*node -e*")
| table _time hostname principal properties.command_line
```

Answer key — staff + self-study

Expected (M5-A1): `gitlab-runner` on `CI-RUNNER-03` running a `curl -fsSL ... | sh` step that isn't in the committed pipeline — the poisoned build.

Model answer (interpretation Q): the tell is a step that isn't in the committed pipeline definition. `curl | sh` on a runner is only malicious relative to what the

Answer key — staff + self-study (cont.)

repo says *should* run — behavior against a baseline, again.

2. Reuse your credential-access hunt (M5-A2, Tier 2): H-D05 taught you the cred-theft shape. Apply it to the runner:

```
index=thrunt sourcetype="evidenceforge:ecar:json" object=PROCESS action=CREATE
(properties.command_line="*169.254.169.254*" OR properties.command_line="*CI_JOB_TOKEN*")
| table _time hostname principal properties.command_line
```

Answer key — staff + self-study

Expected (M5-A2): the runner scraping `CI_JOB_TOKEN` / `AWS_*` env and hitting the
IMDS endpoint (`169.254.169.254`) to steal cloud creds. Same cred-theft shape as
H-D05, transferred from developer secret files to CI runner secrets/IMDS.

3. ★ The euphoric-surprise link — same infrastructure (M5-A3, Tier 3):

Guiding question (decision): *Ask your repo whether this new campaign shares infrastructure with anything you've already hunted. Which prior hunt do you pivot off, and what would "same actor" proof actually look like in the data?*

Ask your repo *"Does this new campaign share infrastructure with anything I've already hunted?"* — the agent pivots off your Campaign-1 C2 hunt (H-D04). (Fallback: `athf similar --hunt H-D04`.) The connective tissue is the **same C2 domain and hosting IP** you confirmed in H-D04:

```
index=thrunt sourcetype="bro:ssl:json" server_name="cdn-packmule-telemetry.net"
| stats count values(id.resp_h) as c2_ip by server_name
```

Same domain, same `203.0.113.44` — present in *both* campaigns' data. Now attribute the client: egress runs through the forward proxy, so Zeek `ssl` shows the proxy (`10.60.20.30`) as the origin — the **proxy log** carries the true client:

```
index=thrunt sourcetype="apache:access:json" source="*proxy_access.log"
server="cdn-packmule-telemetry.net"
| stats count values(user) as users by client
```

The `client` list now contains **both** a `10.60.10.x` workstation (Campaign 1, `SUDOSHIPPING\alice.chmod`) *and* the `10.60.30.13` CI runner (Campaign 2), reaching the **same C2 domain**. That's the proof: **same actor, new battlefield.** Your Campaign-1 C2 hunt just solved half of Campaign 2.

Answer key — staff + self-study

Expected (M5-A3): the `client` list contains **both** a `10.60.10.x` workstation (Campaign 1) *and* `10.60.30.13` CI runner (Campaign 2), beaconing to the **same C2 domain**. That's the proof: **same actor, new battlefield.**

Model answer (decision Q): pivot off H-D04 (your Campaign-1 C2 hunt). "Same actor"

Answer key — staff + self-study (cont.)

proof is the shared C2 domain (`cdn-packmule-telemetry.net`) + hosting IP (`203.0.113.44`) spanning a Campaign-1 host and Campaign-2 runner — your earlier
hunt just solved half of Campaign 2. This is the euphoric moment; let it land.

4. Close it — exfil to cloud storage (M5-A4, Tier 2):

Guiding question (interpretation): *You'll see traffic to S3. One is exfil, one is a legitimate release. What distinguishes them — and why is the red herring here on purpose?*

```
index=thrunt sourcetype="bro:ssl:json" server_name="*.s3.us-west-2.amazonaws.com"
| stats count by id.orig_h server_name
```

Answer key — staff + self-study

Expected (M5-A4): `tar` of `/builds/sudoshipping/fleet-core` → `curl -T` to an
attacker S3 bucket. **Red herring:** `CI-RUNNER-05` legitimately deploys to `sudo-releases.s3 ...` — do NOT flag the real release.
Model answer (interpretation Q): the discriminator is bucket name, committed step,

Answer key — staff + self-study (cont.)

and cadence — the legit deploy goes to a known bucket via a committed pipeline step
on a regular schedule; the exfil goes to an attacker bucket via an uncommitted step. The red herring is deliberate: real environments have benign S3 traffic, and
a hunter who flags the release loses credibility.

WRITE H-D06 (DO THIS EVEN IF YOU ONLY FIND M5-A1)

Guiding question (decision): *What single line in this hunt record is the entire point of the workshop — the thing that made Campaign 2 fast?*

```
athf hunt new \
  --title "CRON GOBLIN CI-runner poisoning (same-actor pivot from packmule campaign)" \
  --technique T1195.003 \
  --tactic initial-access --tactic command-and-control --tactic exfiltration \
  --platform linux \
  --data-source edr --data-source network \
  --actor "CRON GOBLIN" \
  --behavior "malicious injected CI build step steals runner cloud creds, beacons to Campaign-1 C2 infra, exfils source to attacker cloud storage" \
  --location "<the poisoned CI runner you found in step 1/3>; shared C2 cdn-packmule-telemetry.net" \
  --evidence "eCAR curl|sh + IMDS access; Zeek ssl shared domain across workstation + runner" \
  --threat-context "same actor/infra as H-D02/H-D04 C2 hunts; linked via athf similar. New TTP: CI pipeline poisoning." \
  --hunter "<your-name>" \
  --non-interactive
```

Answer key — staff + self-study

Fill `--location` with the runner from steps 1/3: `CI-RUNNER-03 (10.60.30.13)`.
Model answer (decision Q): the `--threat-context` line naming H-D02/H-D04 is the
point of the whole day — your earlier hunts made this one fast. That citation, not
the finding, is what "memory is the edge" means. Target shape:

Answer key — staff + self-study (cont.)

`HUNTS_EXPECTED/H-D06.md`.

VERIFY

```
athf hunt list          # 6 hunts — you leave with the full set
athf hunt coverage       # two campaigns, one actor, mapped
```

THE AGENTIC MOMENT (RECAP)

Guiding question (reflection): *Two hunters walk into Module 5 — one with a documented repo, one without. Trace exactly where their 25 minutes diverge, and why that gap is the whole workshop.*

Manual dead-end: fresh CI telemetry, an unfamiliar battlefield, 25 minutes. The ad-hoc hunter starts cold and never connects the runner's beacon to the earlier campaign — they see two unrelated events.

Agentic recovery (full loop): the hunter with a documented repo runs `athf similar` / `find_similar_hunts` against their Campaign-1 C2 hunt, and the shared infrastructure (M5-A3) links the two campaigns instantly.

Answer key — staff + self-study

Model answer (reflection Q): the ad-hoc hunter re-derives everything from scratch,
sees the runner beacon and the workstation beacon as unrelated, and runs out of
time. The hunter with memory runs `find_similar_hunts` against H-D04, the shared
C2 domain + hosting block links the campaigns in one move (M5-A3), and H-D05's cred-access shape

Answer key — staff + self-study (cont.)

transfers structurally to M5-A2. The framework remembered what you hunted, so the
pivot took minutes instead of the whole session. That gap — between the hunter with
memory and the hunter without — is the entire thesis of the workshop, proven live.

MODULE 5 — BONUS / STRETCH HUNTS

> For the fastest pods, if any finish all four M5 artifacts inside 25 min. See `seeding/TIER_MANIFEST.md`.

BONUS 1 — FULL TWO-CAMPAIGN TIMELINE

Guiding question (decision): *Build one timeline spanning both campaigns. What anchors the two together into a single story a threat-intel team would ship?*

Answer key — staff + self-study

Expected: a single timeline from Campaign-1 install (WS-BUILD-01) through the
CI-runner exfil (CI-RUNNER-03), anchored on the shared C2 domain.
Model answer (decision Q): the shared C2 infrastructure is the anchor — one actor,
one infrastructure, two TTPs, one story. That's the artifact a real threat-intel

Answer key — staff + self-study (cont.)

team ships: not two incidents, one campaign.

BONUS 2 — WRITE THE DETECTION HAND-OFF NOTE

Guiding question (decision): *You've hunted this. What would make it a durable detection? Name the behavior stable enough to alert on — and the ones that aren't.*

Answer key — staff + self-study

Model answer (decision Q): shared-infra beaconing and uncommitted CI steps age well
— they're behavioral and expensive for the actor to change. Hostnames and hashes
don't age well — trivial to rotate. Write one paragraph: alert on the durable behaviors. This is a deliberate teaser for the wrap's ADEF preview — hunting finds

Answer key — staff + self-study (cont.)

it once, detection catches it every time.

BONUS 3 — BREAK THE LINKAGE ON PURPOSE

Guiding question (reflection): *If CRON GOBLIN had rotated to a fresh C2 domain for the CI campaign, would your Campaign-1 repo still have helped? What behavior — not indicator — would still transfer?*

Answer key — staff + self-study

Model answer (reflection Q): yes — the credential-access shape (H-D05) and the
"uncommitted step on a trusted runner" pattern survive infrastructure rotation. Only the shared-domain link (M5-A3) would break; the behavioral hunts still transfer. Articulating this is the deepest version of "behavior over indicators" in

Answer key — staff + self-study (cont.)

the whole workshop — indicators rotate, behavior is the durable pivot.